Generarea automată de teste și date de intrare folosind AVMf și criterii formale de acoperire

Andruță Andra Mihaela

Ciorîță Elena Alexandra

Tunaru Ioana Alexandra

Concepte și fundamente teoretice

Ce este Alternating Variable Method (AVM)?

→ **Definiție:** Este o tehnică de **căutare locală** (local search) utilizată în ingineria software bazată pe căutare (SBSE).

→ **Concept:** Optimizează un vector de variabile luând fiecare variabilă pe rând și aplicându-i un proces de căutare individual.

→ **Eficiență:** Este recunoscută ca fiind o metodă rapidă și eficientă, rezistând în timp chiar și în fața algoritmilor globali mai complecși (precum Algoritmii Genetici).

Cum funcționează? (Etapele căutării)

Algoritmii implementați în AVMf (IPS, GS, PS) urmează în general acești pași:

→ **Mișcări Exploratorii (Exploratory Moves):** Se fac mici modificări (creșteri/descreșteri de +/- 1 sau valoarea minimă admisă) pentru a stabili o direcție favorabilă.

→ **Mișcări de Tip Model (Pattern Moves):** Dacă o direcție este bună, se fac pași tot mai mari în acea direcție până când valoarea funcției obiectiv nu se mai îmbunătățește.

→ **Restartare:** Dacă algoritmul rămâne blocat într-un optim local, căutarea se restartează de la o nouă poziție (adesea aleatorie).

Algoritmi de Căutare în AVMf

AVMf include trei variante principale pentru căutarea variabilelor:

1. **Iterated Pattern Search (IPS):** Metodă de optimizare care combină explorarea locală cu pași mai mari („pattern moves”), reluând căutarea când nu mai apar îmbunătățiri.
2. **Geometric Search (GS):** Folosește căutarea binară pentru a localiza optimul odată ce acesta a fost încadrat între două limite.
3. **Pattern Search (PS):** Metodă fără derivată care explorează vecinătatea unei soluții și ajustează direcția sau pasul în funcție de îmbunătățiri.

Strategii de Căutare Locală în AVMf

Abordarea Comună

Toți cei trei algoritmi folosiți în proiectul nostru funcționează prin alternarea variabilelor dintr-un vector. Aceștia încep cu **mișcări exploratorii** pentru a găsi o direcție de îmbunătățire a funcției obiectiv, urmate de **mișcări de tip model** (pași tot mai mari) pentru a accelera căutarea.

Diferențierea Strategiilor (Ce am folosit noi)

Algoritm	Strategia de Finalizare a Căutării	Caracteristică Cheie
Pattern Search	Se oprește imediat ce o mișcare de tip model eșuează (nu mai îmbunătățește scorul).	Cea mai simplă formă de căutare locală.
Iterated Pattern Search (IPS)	Dacă o mișcare mare eșuează, algoritmul revine la explorarea locală pentru a continua căutarea.	Este implementarea clasică a metodei AVM.
Geometric Search (GS)	Când optimul este depășit, acesta „încadrează” intervalul și aplică o **căutare binară**	**Eficiență sporită:** Găsește valoarea exactă mult mai rapid.

Implementare și Abordări Individuale

Strategia de generare automată (MoveEvent) - Andra

- **Configurația AVMf:** Utilizarea algoritmului **Geometric Search (GS)** pentru optimizarea vectorului de intrare prin pași exponențiali și căutare binară.
- **Definirea Fitness-ului:** Aplicarea principiului **"Lower is Better"**; succesul este definit ca atingerea distanței **0.0** față de ramura țintă.
- **Descompunerea condițiilor:** Pentru a garanta **100% Branch Coverage**, condițiile complexe (operatorul ||) au fost fragmentate în target-uri AVMf individuale (ex: 1T-Left și 1T-Right).
- **Automatizarea fluxului:** Datele generate de AVMf sunt exportate automat în format **CSV** pentru a fi utilizate de suita de teste JUnit.

```java
private static NumericObjectiveValue dist(double distance) {  26 usages  andra2602
    return NumericObjectiveValue.lowerIsBetterObjectiveValue(distance,  optimum: 0.0);
}

private static Vector buildVector() {  1 usage  andra2602
    Vector v = new Vector();
    v.addVariable(new IntegerVariable( initialValue: 2,   min: -1,   max: 5));  // dayEvent
    v.addVariable(new IntegerVariable( initialValue: 0,   min: -3,  max: 10));  // eventIndex
    v.addVariable(new IntegerVariable( initialValue: 1,   min: -1,   max: 5));  // newDay
    return v;

}
```

```java
for (int t = 0; t < objFunctions.length; t++) {
    System.out.println("--- Branch " + branchIds[t] + " ---");
    System.out.println("Target: " + targets[t]);

    Vector vector = buildVector();
    TerminationPolicy tp =
            TerminationPolicy.createMaxEvaluationsTerminationPolicy(1000);
    Initializer initializer = new RandomInitializer(new Well19937c());
    AlternatingVariableMethod avm =
            new AlternatingVariableMethod(new GeometricSearch(), tp, initializer);
    Monitor monitor = avm.search(vector, objFunctions[t]);

    int[] found = extract(vector);
    int result  = MoveEventTestable.moveEventTestable(found[0], found[1], found[2]);
    boolean hit = monitor.getBestObjVal().isOptimal();
    if (hit) targetsHit++;
```

Analiza Rezultatelor — Performanță și Limite (Andra)

- **Criterii Formale Atinse:**

- Branch Coverage: 100% (Toate cele 13 ținte au atins optimul global 0.0).

- Statement Coverage: 100% (derivat din acoperirea tuturor ramurilor de decizie).

- **Indicatori de Performanță (Date Reale):**

- Timp de execuție: **0–1 ms** pentru majoritatea target-urilor (Eficiență maximă).

- Efort de căutare: 5–29 evaluări (**standard**) | până la 137 (**cazuri complexe – B5**)

- **Analiza Limitelor și Soluții:**

- Limita: Căutarea locală poate ignora ramurile "False" ale unui IF.

- Soluție: Am definit ținte specifice pentru ramurile de tip else sau false (ex: 5T-F, 5T-C).

- **Validare:** Datele salvate în **CSV** au fost verificate automat prin suita **JUnit**, confirmând corectitudinea input-urilor generate.

```
--- Branch 1T-L ---
Target: dayEvent < 1 = true                                    ->  zi invalida (stanga)
Best solution: [0, 2, 1]
Best objective value: 0.0
Number of objective function evaluations: 5 (unique: 4)
Running time: 1ms
---

--- Branch 1T-R ---
Target: dayEvent > 3 = true                                    ->  zi invalida (dreapta)
Best solution: [5, 2, 2]
Best objective value: 0.0
Number of objective function evaluations: 6 (unique: 4)
Running time: 0ms
---

--- Branch 2T ---
Target: eventIndex < 0 = true                                  ->  index invalid
Best solution: [3, -2, 4]
Best objective value: 0.0
Number of objective function evaluations: 17 (unique: 9)
Running time: 0ms
---
```

```
✓ MoveEventAVMTest                                                    5 ms
  ✓ Toate inputurile din CSV produc rezultatul asteptat               4 ms
  ✓ return 5 — RESTRICTED_MOVE: newDay==1 && eventIndex>=4 && (dayEvent==3 || eventIndex>5)   1 ms
  ✓ return 3 — INVALID_NEW_DAY: newDay < 1 || newDay > 3
  ✓ return 4 — SAME_DAY: newDay == dayEvent
  ✓ return 2 — INVALID_INDEX: eventIndex < 0
  ✓ return 1 — INVALID_DAY: dayEvent < 1 || dayEvent > 3
  ✓ return 0 — MOVE_SUCCESS: toate conditiile valide
```

Element	Class, %	Method, %	Line, %	Branch, %
avmf.testable	100% (3/3)	100% (5/5)	100% (44/44)	100% (68/68)
BuyTicketTestable	100% (1/1)	100% (2/2)	100% (21/21)	100% (30/30)
MoveEventTestable	100% (1/1)	100% (1/1)	100% (6/6)	100% (20/20)
SwapOrganizersTestable	100% (1/1)	100% (2/2)	100% (17/17)	100% (18/18)

Generare automată cu IteratedPatternSearch()

- **Branch-uri:** Condițiile sunt descompuse în ramuri (true/false).
- **Target-uri:** Pentru fiecare branch se testează atât cazul în care condiția este adevărată, cât și falsă.
- **Fitness:** Pentru fiecare branch se construiește o funcție de fitness care măsoară distanța față de target **și include condițiile anterioare pentru a ghida execuția corectă.**
- **Optimizare:** „Lower is Better” → fitness 0.0 = succes.
- **Search:** Se folosește **Iterated Pattern Search (IPS).**
- **Generare:** Inputurile sunt ajustate iterativ până satisfac condiția.
- **Output:** Inputurile generate sunt salvate în **CSV** și folosite în **JUnit.**

```
// Branch 11 – dayEvent < 1
// Branch 12 – dayEvent > 3
// Branch 2  – eventIndex < 0
// Branch 31 – newDay < 1
// Branch 32 – newDay > 3
// Branch 4  – newDay == dayEvent
// Branch 5  – succes
```

```
// rulez AVMf pentru fiecare ramura si pentru true/false
int[] branches = {11, 12, 2, 31, 32, 4, 5};
for (int branch : branches) {
    for (boolean targetTrue : new boolean[]{true, false}) {
        if (branch == 5 && !targetTrue) continue;
        runForBranch(branch, targetTrue, csvWriter);
    }
}
```

```
} else if (branchID == 2) {
    // trebuie dayEvent valid: 1 <= dayEvent <= 3
    distance += Math.max(0, 1 - dayEvent);
    distance += Math.max(0, dayEvent - 3);
    // if (eventIndex < 0)
    if (targetTrue) {
        distance += Math.max(0, eventIndex + 1);
    } else {
        distance += Math.max(0, -eventIndex);
    }
```

Analiza rezultatelor

- **Verificare corectitudine:** Verificăm că metoda returnează rezultatul așteptat pentru fiecare input generat.
- **Verificare acoperire:** Verificăm că fiecare input ajunge exact în ramura țintă, nu doar produce un rezultat corect.
- **Performanță:** 1–86 evaluări, în funcție de complexitate.
- **Eficiență:** Timp de execuție foarte mic (0–6 ms)
- **Convergență:** Fitness = 0 pentru toate target-urile.
- **Branch Coverage: 100% branch coverage** (prin descompunerea condițiilor)

```
--- Branch 11T ---
Target: dayEvent < 1 = true → zi eveniment prea mică
Best solution: [0, -5, 9]
Best objective value: 0.0
Number of objective function evaluations: 4 (unique: 2)
Running time: 0ms
--- Branch 11F ---
Target: dayEvent < 1 = false → dayEvent >= 1
Best solution: [4, 16, -3]
Best objective value: 0.0
Number of objective function evaluations: 7 (unique: 5)
Running time: 0ms
```

```
int actualResult = MoveEventTestable.moveEventTestable(
        dayEvent, eventIndex, newDay);

assertEquals(
        message: "Branch " + branchId + (targetTrue ? "T" : "F") +
                " – rezultat gresit pentru input: " + line,
        expectedResult,
        actualResult
);
```

```
switch (branchId) {
    case 11:
        if (targetTrue) {
            assertEquals( message: "Branch 11T trebuie să returneze 1",  expected: 1, actualResult);
        } else {
            assertTrue( message: "Branch 11F trebuie să treacă de validarea dayEvent < 1",
                    condition: dayEvent >= 1);
        }
        break;
```

Generare automată cu PatternSearch

- Se analizează metoda `MoveEventTestable` și se identifică toate branch-urile.
- Pentru fiecare branch se definește o **funcție de fitness**.
- Se generează valori inițiale aleator pentru input.
- Se aplică **Pattern Search** pentru a modifica iterativ valorile.
- Fitness-ul este evaluat la fiecare pas și ghidează căutarea.
- Algoritmul converge către inputuri care satisfac condiția target.
- Inputurile găsite sunt salvate automat în fișier.
- Se generează teste **JUnit** pe baza acestor inputuri.
- Testele sunt rulate pentru a verifica corectitudinea și acoperirea.

```java
public static int moveEventTestable(  8 usages  ▲ al
        int dayEvent, int eventIndex, int newDay
) {
    if (dayEvent < 1 || dayEvent > 3) return 1;
    if (eventIndex < 0) return 2;
    if (newDay < 1 || newDay > 3) return 3;
    if (newDay == dayEvent) return 4;
    return 0;
}
```

Metoda testată

```java
// ---- BRANCH 11: day < 1 ----
if (targetBranch == 11) {
    fitness += Math.max(0, day);
}

// ---- BRANCH 12: day > 3 ----
if (targetBranch == 12) {
    fitness += Math.max(0, 3 - day);
}
```

Exemplu de fitness

Analiza rezultatelor

Branch Coverage Results

- Pentru fiecare branch a fost găsit un set valid de inputuri.
- Algoritmul converge rapid (maxim 10 evaluări în majoritatea cazurilor).
- Fiecare soluție satisface exact condiția target (ex: index negativ, zi invalidă etc.).
- Testele au fost generate automat pe baza acestor inputuri.

```
--- Branch 12 ---
Target: dayEvent > 3 = true → zi eveniment prea mare
Best solution: [4, 135, 5]
Number of objective function evaluations: 5

--- Branch 2 ---
Target: eventIndex < 0 = true → index invalid
Best solution: [3, -11, 1]
Number of objective function evaluations: 3
```

```java
public static int moveEventTestable(  8 usages  alele
        int dayEvent, int eventIndex, int newDay
) {
    if (dayEvent < 1 || dayEvent > 3) return 1;
    if (eventIndex < 0) return 2;
    if (newDay < 1 || newDay > 3) return 3;
    if (newDay == dayEvent) return 4;
    return 0;
}
```

Achieving 100% Coverage

- A fost obținut 100% branch coverage.
- Toate ramurile metodei au fost executate cel puțin o dată.
- Nu există cod neacoperit.

```java
public class MoveEventGeneratedTests {  alelena14 *

    @Test  alelena14 *
    void test_0() { assertEquals( expected: 1, MoveEventTestable.moveEventTestable( dayEvent: 0,  eventIndex: 83,  newDay: -1)); }

    @Test  alelena14 *
    void test_1() { assertEquals( expected: 1, MoveEventTestable.moveEventTestable( dayEvent: 5,  eventIndex: -17,  newDay: 4)); }
```

Abordări diferite în utilizarea AVMf

ANDRA
~BuyTicket~

Outcome-Driven Targeting

- **Orientare pe Stări Finale:** Generarea inputurilor bazată pe statusurile returnate (**BuyTicketStatus**), fiecare fiind tratat ca un punct optim în căutare
- **Ierarhizarea Condițiilor:** Funcțiile de tip *branch distance* (ex: **distPass**) ghidează algoritmul prin etapele de validare până la atingerea ramurilor dorite.
- **Validare Hibridă:** Implementarea unui flux complet de testare: **Generare AVMf → Persistență CSV → Verificare JUnit**, asigurând consistența între datele generate și logica de business.

ALEXANDRA T
~ParticipantSanction~

- **Branch-Driven Targeting:** Fiecare branch este tratat ca un target separat (true/false), cu o funcție obiectiv dedicată.
- **Fitness ghidat de condiții anterioare:** Fitness-ul calculează distanța față de ramura țintă și impune condițiile anterioare pentru a ghida corect AVMf.
- **Pipeline automat de testare:** Soluțiile găsite sunt salvate în CSV și verificate ulterior automat prin teste JUnit.

ALEXANDRA C
~ReserveSeat~

- **Outcome Targeting**: Testarea este ghidată de rezultatele dorite (branch-uri). Fiecare rezultat este tratat ca un obiectiv de optimizare.
- **Branch Distance & Fitness Function**: Funcția de fitness calculează distanța față de condițiile target. Pattern Search ajustează inputurile pentru a minimiza această distanță.
- **Pipeline testare**: Inputurile sunt salvate și transformate automat în teste JUnit. Se obține un proces complet automatizat de testare.

CONCLUZII

1. Eficiența AVMf în practică

- **Rapiditate:** Algoritmul converge extrem de rapid pe variabile de tip int și fixed-point, majoritatea testelor fiind generate în mai puțin de 10ms.
- **Cost Computațional Redus:** Numărul mic de evaluări unice ale funcției obiectiv confirmă eficiența strategiilor de căutare locală față de căutările aleatorii sau globale.

2. Lecții învățate și limite identificate

- **Dependența de Funcția Obiectiv:** Calitatea datelor generate depinde critic de modul în care este definită „distanța de ramură” (branch distance).
- **Problema Optimului Local:** În peisaje de căutare complexe (precum regex sau condiții cu multe ramuri else), algoritmul poate necesita restartări multiple pentru a găsi soluția globală.
- **Abordarea Hibridă:** Cea mai bună metodă de a asigura un coverage de 100% este combinarea **generării automate** cu **analiza valorilor de frontieră** (Boundary Value Analysis) și teste manuale de validare.

3. Impactul proiectului

- Framework-ul a permis transformarea unei sarcini manuale repetitive (scrierea datelor de test) într-un proces automatizat, precis și ușor de verificat prin JUnit

Vă mulțumim!

Întrebări?